

ELE440 Real-Time Embedded Systems

Final Assignment

Two-Axis Tilt Switch with Audio Feedback

Using CMSIS-RTOS RTX on STM32F4Discovery

James Bray

Registration: 220123839

January 16, 2026

Part A.I: Code Functionality and Design

This report describes a two-axis tilt switch with pause mode and audio feedback running CMSIS-RTOS RTX on the STM32F4Discovery board. The system monitors board orientation via a three-axis accelerometer and responds with visual (LEDs) and audio feedback. It also features a pause mode. The design meets all brief requirements (Levels 1-3) and includes an out-of-scope Level 4 musical feature.

Overall Functionality

Four RTOS threads manage the system, each operating at 50ms cycles to balance response times and processing times. The tilt thread polls the LIS3DSH accelerometer via SPI1, extracting X, Y, Z data and computing magnitude of the tilt. The button thread monitors PA0 for press events, implementing 200ms debouncing for robustness. The LED thread reads the current tilt magnitude and application mode, updating GPIO pins to light LEDs to show tilt direction. The audio thread covers the audio DAC, triggering notes to play based on current magnitude and muting output when not needed.

Threads use volatile shared variables with a single-writer pattern, eliminating the need for mutexes: `current_mode` (Button writes only), `accel_x/y/z` (Tilt writes only), `tilt_magnitude`, `tilt_period`, and `audio_muted` (LED writes only). This design means threads are safe from locking, which is critical before the RTOS starts up and synchronisation isn't possible. Each thread calls `osDelay(50)` to yield the CPU, allowing the RTOS scheduler to run other threads without busy-waiting.

Level 1: Tilt Detection

The LIS3DSH accelerometer communicates using SPI1 (Mode 3, 2.625MHz) and is configured for $\pm 2g$ full-scale range and 25Hz output data rate, which is just right to ensure responsiveness without too much flickering. The tilt thread samples the accelerometer every 50ms (20Hz software polling), reading 16-bit signed integer values from the X, Y, and Z output registers. At 2g scale, each LSB represents $\approx 61 \mu g$, so a deadzone threshold of 2857 LSB corresponds to approximately 10° of tilt.

The LED mapping implements two-axis tilt detection: red (PD14) illuminates when $a_x > +2857$ LSB (board tilted right), green (PD12) when $a_x < -2857$ LSB (tilted left), orange (PD13) when $a_y > +2857$ LSB (tilted toward USB connector), and blue (PD15) when $a_y < -2857$ LSB (tilted away from USB). At rest (both axes within deadzone), all LEDs are off. When tilted diagonally, multiple LEDs turn on as per the requirements.

Level 2: Pause Mode

The button thread monitors PA0 (user button), which toggles the system mode through a three-state machine: `MODE_TILT` (normal operation), `MODE_PAUSE_ENTRY` (transition state), and `MODE_PAUSE` (sustained pause). In `MODE_TILT`, the LED thread directly maps tilt values to LED outputs as described above. Then when pressed, the system enters `MODE_PAUSE_ENTRY`, where the LED thread lights all four LEDs for a second, then turns them off for a second. After this sequence, the system transitions to `MODE_PAUSE`, where only the red LED (PD14) blinks continuously at 1Hz (500ms on, 500ms off) until the next button click returns to `MODE_TILT`.

Level 3: Audio Feedback

The CS43L22 audio codec (I2C1 at 0x4A, reset PD4) streams samples to the speaker via SPI3 I2S at 48kHz sample rate. Initialisation uses direct register access (no HAL timeouts) for robustness before the RTOS startup. The software then generates a triangle wave to output at a certain frequency, which is constant for pause mode, but variable in tilt mode (see Level 4). The SPI3 I2S peripheral is configured for 16-bit samples, Master mode, and 48kHz frequency with I2S standard frame format. Triangle wave samples are generated in the SPI3 interrupt handler, ensuring timing accuracy without extra load on the Audio Thread.

In pause mode, the audio beeps are synchronised with the red LED blink. The LED thread controls both the LED state and the `audio_muted` flag on the same 50ms cycle, ensuring the beep sounds when the red LED is on. This synchronisation is done with the shared `audio_muted` variable, which the Audio Thread reads to gate the triangle wave output.

Robustness and Efficiency

Robustness: Direct SPI/I2C register access (no HAL) avoids hanging before RTOS kernel startup, ensuring everything initialise reliably. `OS_CLOCK` is set to 168MHz to ensure correct `osDelay()` timing as mentioned in the assignment. All GPIO outputs use atomic `BSRR` (Bit Set/Reset Register) writes, preventing race conditions when multiple threads update different LED pins concurrently. The accelerometer `WHO_AM_I` register is checked during initialisation to ensure its operation. Thread priorities were set based on task urgency: the button thread runs at ‘Above Normal’ priority to detect user input instantly, Tilt and LED threads run at ‘Normal’ priority, and the Audio Thread at ‘Below Normal’ priority, as audio generation isn’t critical to the core functionality.

Efficiency: Threads yield the CPU by using `osDelay(50)`, allowing the RTOS scheduler to interleave execution and prevent busy-waiting. SPI1 clock is set to 2.625MHz (APB2 prescaler 32), ensuring reliable communication with the accelerometer datasheet requirement for Mode 3 (`CPOL=1`, `CPHA=1`). I2C1 clock is set to 100kHz standard mode for robust audio codec communication. The accelerometer 25Hz ODR setting (40ms period) is paired with 20Hz software polling (50ms thread cycle), ensuring no samples are missed. No dynamic memory allocation is used; all data structures are statically declared, eliminating heap fragmentation and `malloc()` unpredictability in a real-time system. Direct register code avoids HAL abstraction overhead, reducing interrupt latency and code size.

Level 4: Scaling Up (out of scope, unmarked)

This feature turns the STM32F4Discovery into a musical instrument. By mapping orientation magnitude to a musical scale and synthesising a triangle waveform at the selected frequency, the board plays different notes at different angles. The frequency is selected based on `tilt_magnitude` in proportion to the maximum tilt angle, this outputs glissando as the board is tilted. There are three scales in `thread.c` lines 52–72, feel free to uncomment and see how they sound (blues is my favourite).

Part A.II: Source Code Files

main.c

```
1 // =====
2 // cmsis-rtos main - initialisation and startup
3 // =====
4
5 #include "stm32f4xx.h"
6 #include "cmsis_os.h"
7 #include "my_defines.h"
8 #include "Thread.h"
9
10 int main(void) {
11
12     // os and system init
13     osKernelInitialize();
14     SystemCoreClockUpdate();
15
16     // peripheral init
17     Init_LEDs();
18     Init_Button();
19     Init_SPI();
20     Init_Accelerometer();
21     Init_Audio();
22
23     // thread init
24     Init_Tilt_Thread();
25     Init_Button_Thread();
26     Init_LED_Thread();
27     Init_Audio_Thread();
28
29     // start os
30     osKernelStart();
31
32     while (1) {}
33 }
```

my_defines.h

```
1 // =====
2 // my_defines.h - definitions and function declarations
3 // =====
4
5 #ifndef MY_DEFINES_H
6 #define MY_DEFINES_H
7
8 #include "stm32f4xx.h"
9
10 // =====
11 // constants
```

```

12 // =====
13
14 // led pin definitions
15 // ref: STM32F4DISCOVERY User Manual, section 4.4, p.16
16 #define LED_GREEN          12
17 #define LED_ORANGE         13
18 #define LED_RED            14
19 #define LED_BLUE           15
20
21 // accelerometer definitions (lis3dsh)
22 // ref: LIS3DSH datasheet, section 7 (Register Map), p.29-31
23 #define LIS3DSH_WHO_AM_I    0x0F
24 #define LIS3DSH_CTRL_REG4   0x20
25 #define LIS3DSH_CTRL_REG5   0x24
26 #define LIS3DSH_STATUS      0x27
27 #define LIS3DSH_OUT_X_L     0x28
28 #define LIS3DSH_OUT_X_H     0x29
29 #define LIS3DSH_OUT_Y_L     0x2A
30 #define LIS3DSH_OUT_Y_H     0x2B
31 #define LIS3DSH_OUT_Z_L     0x2C
32 #define LIS3DSH_OUT_Z_H     0x2D
33 #define LIS3DSH_READ         0x80
34 #define LIS3DSH_ID           0x3F
35
36 // tilt detection
37 #define TILT_DEADZONE        2857
38
39 // timing (ms)
40 #define PAUSE_BLINK_ALL_MS   1000
41 #define PAUSE_BLINK_RED_MS   500
42 #define ACCEL_SAMPLE_PERIOD_MS 50
43 #define BUTTON_DEBOUNCE_MS   50
44
45 // audio
46 #define PAUSE_TONE_FREQ_HZ    110
47 #define BEEP_SAMPLE_RATE      48000
48 #define AUDIO_SIG_START       0x01
49 #define AUDIO_SIG_STOP        0x02
50
51 // cs43l22 audio dac
52 // ref: CS43L22 datasheet, section 6 (Register Map), p.36-47
53 #define CS43L22_ADDR          0x4A
54 #define CS43L22_RESET_PIN     4
55 #define CS43L22_CHIP_ID       0x01
56 #define CS43L22_PWR_CTRL1     0x02
57 #define CS43L22_PWR_CTRL2     0x04
58 #define CS43L22_CLK_CTRL      0x05
59 #define CS43L22_IFACE_CTRL1    0x06
60 #define CS43L22_IFACE_CTRL2    0x07
61 #define CS43L22_PASSTHROUGH    0x08
62 #define CS43L22_ANALOG_SR      0x0A

```

```

63 #define CS43L22_PLAYBACK1      0x0D
64 #define CS43L22_MISC_CTRL      0x0E
65 #define CS43L22_PLAYBACK2      0x0F
66 #define CS43L22_PCM_VOL_A      0x1A
67 #define CS43L22_PCM_VOL_B      0x1B
68 #define CS43L22_BEEP_FREQ      0x1C
69 #define CS43L22_BEEP_VOL       0x1D
70 #define CS43L22_BEEP_CFG       0x1E
71
72 // =====
73 // function declarations
74 // =====
75
76 // init functions
77 void Init_LEDs(void);
78 void Init_Button(void);
79 void Init_SPI(void);
80 void Init_Accelerometer(void);
81 void Init_I2C(void);
82 void Init_I2S(void);
83 void Init_Audio(void);
84
85 // led control functions
86 void LED_On(uint8_t led_pin);
87 void LED_Off(uint8_t led_pin);
88 void LED_Toggle(uint8_t led_pin);
89 void LEDs_All_On(void);
90 void LEDs_All_Off(void);
91
92 // button functions
93 uint8_t Button_Read(void);
94 uint8_t Button_Clicked(void);
95
96 // accelerometer functions
97 uint8_t SPI_Read_Register(uint8_t reg_addr);
98 void SPI_Write_Register(uint8_t reg_addr, uint8_t value);
99 void Read_Accelerometer(int16_t *x, int16_t *y, int16_t *z);
100
101 // tilt led update
102 void Update_Tilt_LEDs(int16_t accel_x, int16_t accel_y);
103
104 // audio functions
105 void Audio_Send_Sample(void);
106 void Audio_Set_Mute(uint8_t mute);
107 void Audio_I2C_Write(uint8_t reg, uint8_t value);
108
109 // =====
110 // global handles
111 // =====
112
113 extern SPI_HandleTypeDef SPI_Handle;

```

```

114 extern I2C_HandleTypeDef I2C_Handle;
115
116 #endif

```

my_defines.c

```

1  // =====
2  // my_defines.c - library functions
3  // =====
4
5  #include "stm32f4xx.h"
6  #include "my_defines.h"
7
8  static uint8_t button_prev_state = 0; // state for button click detection
9
10 // =====
11 // led functions - covered in course labs, no comments needed
12 // =====
13
14 void Init_LEDs(void) {
15     RCC->AHB1ENR |= RCC_AHB1ENR_GPIODEN;
16
17     GPIOD->MODER |= GPIO_MODER_MODER12_0;
18     GPIOD->MODER |= GPIO_MODER_MODER13_0;
19     GPIOD->MODER |= GPIO_MODER_MODER14_0;
20     GPIOD->MODER |= GPIO_MODER_MODER15_0;
21
22     LEDs_All_Off();
23 }
24
25 void LED_On(uint8_t led_pin) {
26     GPIOD->BSRR = (1 << led_pin);
27 }
28
29 void LED_Off(uint8_t led_pin) {
30     GPIOD->BSRR = (1 << (led_pin + 16));
31 }
32
33 void LED_Toggle(uint8_t led_pin) {
34     GPIOD->ODR ^= (1 << led_pin);
35 }
36
37 void LEDs_All_On(void) {
38     GPIOD->BSRR = (1 << LED_GREEN) | (1 << LED_ORANGE) |
39                 (1 << LED_RED) | (1 << LED_BLUE);
40 }
41
42 void LEDs_All_Off(void) {
43     GPIOD->BSRR = (1UL << (LED_GREEN + 16)) |
44                 (1UL << (LED_ORANGE + 16)) |

```

```

45         (1UL << (LED_RED + 16)) |
46         (1UL << (LED_BLUE + 16));
47     }
48
49     // =====
50     // button functions - covered in course labs, no comments needed
51     // =====
52
53     void Init_Button(void) {
54         RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
55     }
56
57     uint8_t Button_Read(void) {
58         if (GPIOA->IDR & GPIO_IDR_IDR_0) { return 1; }
59         else { return 0; }
60     }
61
62     uint8_t Button_Clicked(void) {
63         uint8_t current = Button_Read();
64         uint8_t clicked = 0;
65
66         if (button_prev_state == 1 && current == 0) { clicked = 1; }
67
68         button_prev_state = current;
69         return clicked;
70     }
71
72     // =====
73     // spi functions
74     // =====
75
76     static uint8_t SPI_TransferByte(uint8_t data) {
77         while (!(SPI1->SR & SPI_SR_TXE)) {} // wait until tx buffer empty
78         SPI1->DR = data; // send byte
79         while (!(SPI1->SR & SPI_SR_RXNE)) {} // wait until not empty
80         return (uint8_t)SPI1->DR; // read byte
81     }
82
83     void Init_SPI(void) {
84         volatile int i;
85
86         // enable spi1 and gpioa, gpioe clocks
87         RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;
88         RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
89         RCC->AHB1ENR |= RCC_AHB1ENR_GPIOEEN;
90
91         // pa5 (sck)
92         GPIOA->MODER &= ~(3UL << (5 * 2)); // clear mode
93         GPIOA->MODER |= (2UL << (5 * 2)); // alt func mode
94         GPIOA->AFR[0] &= ~(0xFUL << (5 * 4)); // clear af
95         GPIOA->AFR[0] |= (5UL << (5 * 4)); // af5 (spi1)

```



```

96     GPIOA->OSPEEDR |= (2UL << (5 * 2));    // med speed
97
98     // pa6 (miso)
99     GPIOA->MODER &= ~(3UL << (6 * 2));      // clear mode
100    GPIOA->MODER |= (2UL << (6 * 2));        // alt func mode
101    GPIOA->AFR[0] &= ~(0xFUL << (6 * 4));    // clear af
102    GPIOA->AFR[0] |= (5UL << (6 * 4));        // af5 (spi1)
103    GPIOA->OSPEEDR |= (2UL << (6 * 2));      // med speed
104
105    // pa7 (mosi)
106    GPIOA->MODER &= ~(3UL << (7 * 2));      // clear mode
107    GPIOA->MODER |= (2UL << (7 * 2));        // alt func mode
108    GPIOA->AFR[0] &= ~(0xFUL << (7 * 4));    // clear af
109    GPIOA->AFR[0] |= (5UL << (7 * 4));        // af5 (spi1)
110    GPIOA->OSPEEDR |= (2UL << (7 * 2));      // med speed
111
112    // pe3 (cs)
113    GPIOE->MODER &= ~(3UL << (3 * 2));      // clear mode
114    GPIOE->MODER |= (1UL << (3 * 2));        // general output mode
115    GPIOE->OTYPER &= ~(1UL << 3);           // push-pull
116    GPIOE->OSPEEDR |= (2UL << (3 * 2));      // med speed
117    GPIOE->PUPDR |= (1UL << (3 * 2));        // pull-up
118
119    GPIOE->BSRR = (1UL << 3);
120
121    // spi1 config: mode 3 (cpol=1, cpha=1)
122    SPI1->CR1 = 0;
123    SPI1->CR1 |= SPI_CR1_CPOL;               // clock polarity high
124    SPI1->CR1 |= SPI_CR1_CPHA;               // clock phase 2nd edge
125    SPI1->CR1 |= SPI_CR1_MSTR;               // master mode
126    SPI1->CR1 |= SPI_CR1_SSM;               // software slave mgmt
127    SPI1->CR1 |= SPI_CR1_SSI;               // internal slave select
128    SPI1->CR1 |= (4UL << 3);                // baud prescaler: fPCLK/16
129    SPI1->CR1 |= SPI_CR1_SPE;               // enable spi1
130
131    for (i = 0; i < 100; i++) {}            // short delay
132 }
133
134 uint8_t SPI_Read_Register(uint8_t reg_addr) {
135     uint8_t rx_data;
136
137     // send read command and read data
138     GPIOE->BSRR = (1UL << (3 + 16));
139     SPI_TransferByte(LIS3DSH_READ | reg_addr);
140     rx_data = SPI_TransferByte(0x00);
141     GPIOE->BSRR = (1UL << 3);
142
143     return rx_data;
144 }
145
146 void SPI_Write_Register(uint8_t reg_addr, uint8_t value) {

```

```

147
148 // send write command and data
149 GPIOE->BSRR = (1UL << (3 + 16));
150 SPI_TransferByte(reg_addr);
151 SPI_TransferByte(value);
152 GPIOE->BSRR = (1UL << 3);
153 }
154
155 // =====
156 // accelerometer functions
157 // =====
158
159 void Init_Accelerometer(void) {
160
161     // check who_am_i register
162     uint8_t who_am_i;
163     who_am_i = SPI_Read_Register(LIS3DSH_WHO_AM_I);
164     if (who_am_i != LIS3DSH_ID) { return; }
165
166     // ctrl_reg4: 25hz odr, xyz enabled
167     SPI_Write_Register(LIS3DSH_CTRL_REG4, 0x37);
168 }
169
170 void Read_Accelerometer(int16_t *x, int16_t *y, int16_t *z) {
171     uint8_t xl, xh, yl, yh, zl, zh;
172
173     // read x, y, z low and high bytes
174     xl = SPI_Read_Register(LIS3DSH_OUT_X_L);
175     xh = SPI_Read_Register(LIS3DSH_OUT_X_H);
176     yl = SPI_Read_Register(LIS3DSH_OUT_Y_L);
177     yh = SPI_Read_Register(LIS3DSH_OUT_Y_H);
178     zl = SPI_Read_Register(LIS3DSH_OUT_Z_L);
179     zh = SPI_Read_Register(LIS3DSH_OUT_Z_H);
180
181     // combine high and low bytes
182     *x = (int16_t)((xh << 8) | xl);
183     *y = (int16_t)((yh << 8) | yl);
184     *z = (int16_t)((zh << 8) | zl);
185 }
186
187 // =====
188 // tilt logic
189 // =====
190
191 void Update_Tilt_LEDs(int16_t accel_x, int16_t accel_y) {
192     LEDs_All_Off();
193
194     // turn on leds based on tilt direction
195     if (accel_x > TILT_DEADZONE) { LED_On(LED_RED); }
196     if (accel_y > TILT_DEADZONE) { LED_On(LED_ORANGE); }
197     if (accel_x < -TILT_DEADZONE) { LED_On(LED_GREEN); }

```

```

198     if (accel_y < -TILT_DEADZONE) { LED_On(LED_BLUE); }
199 }
200
201 // =====
202 // audio functions
203 // =====
204
205 static volatile uint16_t tone_phase = 0; // phase accumulator
206 extern volatile uint8_t audio_muted;    // audio mute flag
207 extern volatile uint8_t tilt_magnitude; // tilt-based magnitude
208 extern volatile uint16_t tilt_period;    // tilt-based period
209
210 #define TONE_AMPLITUDE 0x4000          // triangle wave amplitude
211
212 static void I2C_Write_Register(uint8_t reg, uint8_t value) {
213     volatile uint32_t dummy;    // dummy read variable
214     volatile uint32_t timeout;  // timeout counter
215
216     // timeouts are used to avoid getting stuck if i2c bus hangs
217
218     // start i2c write sequence
219     timeout = 100000;
220     while ((I2C1->SR2 & I2C_SR2_BUSY) && timeout--) {}
221     if (timeout == 0) return;
222
223     // send start condition
224     I2C1->CR1 |= I2C_CR1_START;
225     timeout = 100000;
226     while (!(I2C1->SR1 & I2C_SR1_SB) && timeout--) {}
227     if (timeout == 0) return;
228
229     // send address and register
230     I2C1->DR = (CS43L22_ADDR << 1);
231     timeout = 100000;
232     while (!(I2C1->SR1 & I2C_SR1_ADDR) && timeout--) {}
233     if (timeout == 0) return;
234     dummy = I2C1->SR2;
235
236     // send register and value
237     I2C1->DR = reg;
238     timeout = 100000;
239     while (!(I2C1->SR1 & I2C_SR1_TXE) && timeout--) {}
240     if (timeout == 0) return;
241
242     // send value
243     I2C1->DR = value;
244     timeout = 100000;
245     while (!(I2C1->SR1 & I2C_SR1_TXE) && timeout--) {}
246     if (timeout == 0) return;
247
248     // send stop condition

```

```

249     I2C1->CR1 |= I2C_CR1_STOP;
250 }
251
252 void Init_I2C(void) {
253     volatile int i;
254
255     // enable i2c1 and gpiob clocks
256     RCC->APB1ENR |= RCC_APB1ENR_I2C1EN;
257     RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;
258
259     // pb6 (scl)
260     GPIOB->MODER &= ~(3UL << (6 * 2)); // clear mode
261     GPIOB->MODER |= (2UL << (6 * 2)); // alt func mode
262     GPIOB->AFR[0] &= ~(0xFUL << (6 * 4)); // clear af
263     GPIOB->AFR[0] |= (4UL << (6 * 4)); // af4 (i2c1)
264     GPIOB->OTYPER |= (1UL << 6); // open drain
265     GPIOB->OSPEEDR |= (1UL << (6 * 2)); // low speed
266     GPIOB->PUPDR |= (1UL << (6 * 2)); // pull-up
267
268     // pb9 (sda)
269     GPIOB->MODER &= ~(3UL << (9 * 2)); // clear mode
270     GPIOB->MODER |= (2UL << (9 * 2)); // alt func mode
271     GPIOB->AFR[1] &=
272         ~(0xFUL << ((9-8) * 4)); // clear af
273     GPIOB->AFR[1] |= (4UL << ((9-8) * 4)); // af4 (i2c1)
274     GPIOB->OTYPER |= (1UL << 9); // open drain
275     GPIOB->OSPEEDR |= (1UL << (9 * 2)); // low speed
276     GPIOB->PUPDR |= (1UL << (9 * 2)); // pull-up
277
278     // configure i2c: 100khz with apb1 42mhz
279     I2C1->CR1 &= ~I2C_CR1_PE; // disable i2c
280     I2C1->CR2 = 42; // apb1 freq in mhz
281     I2C1->CCR = 210; // 100khz clk
282     I2C1->TRISE = 43; // max rise time
283     I2C1->CR1 |= I2C_CR1_PE; // enable i2c
284
285     for (i = 0; i < 100; i++) {} // short delay
286 }
287
288 void Init_I2S(void) {
289     volatile int i;
290
291     // enable spi3 and gpioc, gpioa clocks
292     RCC->APB1ENR |= RCC_APB1ENR_SPI3EN;
293     RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;
294     RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
295
296     // plli2s for 48khz
297     RCC->PLLI2SCFGR = (3UL << 28) | (258UL << 6);
298     RCC->CR |= RCC_CR_PLLI2SON;
299     while (!(RCC->CR & RCC_CR_PLLI2SRDY)) {}

```

```

300
301 // pc7 (mck)
302 GPIOC->MODER &= ~(3UL << (7 * 2)); // clear mode
303 GPIOC->MODER |= (2UL << (7 * 2)); // alt func mode
304 GPIOC->AFR[0] &= ~(0xFUL << (7 * 4)); // clear af
305 GPIOC->AFR[0] |= (6UL << (7 * 4)); // af6 (spi3)
306 GPIOC->OSPEEDR |= (3UL << (7 * 2)); // high speed
307
308 // pc10 (sck)
309 GPIOC->MODER &= ~(3UL << (10 * 2)); // clear mode
310 GPIOC->MODER |= (2UL << (10 * 2)); // alt func mode
311 GPIOC->AFR[1] &= ~(0xFUL << ((10-8) * 4)); // clear af
312 GPIOC->AFR[1] |= (6UL << ((10-8) * 4)); // af6 (spi3)
313 GPIOC->OSPEEDR |= (3UL << (10 * 2)); // high speed
314
315 // pc12 (sd)
316 GPIOC->MODER &= ~(3UL << (12 * 2)); // clear mode
317 GPIOC->MODER |= (2UL << (12 * 2)); // alt func mode
318 GPIOC->AFR[1] &= ~(0xFUL << ((12-8) * 4)); // clear af
319 GPIOC->AFR[1] |= (6UL << ((12-8) * 4)); // af6 (spi3)
320 GPIOC->OSPEEDR |= (3UL << (12 * 2)); // high speed
321
322 // pa4 (ws)
323 GPIOA->MODER &= ~(3UL << (4 * 2)); // clear mode
324 GPIOA->MODER |= (2UL << (4 * 2)); // alt func mode
325 GPIOA->AFR[0] &= ~(0xFUL << (4 * 4)); // clear af
326 GPIOA->AFR[0] |= (6UL << (4 * 4)); // af6 (spi3)
327 GPIOA->OSPEEDR |= (3UL << (4 * 2)); // high speed
328
329 SPI3->I2SCFGR = 0; // disable i2s
330 SPI3->I2SPR = 0; // reset prescaler
331
332 // i2s prescaler for 48khz
333 SPI3->I2SPR = (1UL << 9) | (1UL << 8) | 3; // odd, div=3
334
335 // i2s master transmit, 16-bit
336 SPI3->I2SCFGR = (1UL << 11) | (2UL << 8); // mstr, tx, 16-bit
337 SPI3->I2SCFGR |= (1UL << 10); // i2s enable
338
339 // prime with silence
340 for (i = 0; i < 100; i++) {
341     while (!(SPI3->SR & SPI_SR_TXE)) {}
342     SPI3->DR = 0;
343 }
344
345 // enable txe interrupt
346 SPI3->CR2 |= SPI_CR2_TXEIE;
347 NVIC_SetPriority(SPI3_IRQn, 14);
348 NVIC_EnableIRQ(SPI3_IRQn);
349 }
350

```

```

351 void Init_Audio(void) {
352     volatile int delay;
353
354     // reset pin (pd4)
355     RCC->AHB1ENR |= RCC_AHB1ENR_GPIODEN;
356     GPIOD->MODER &= ~(3UL << (CS43L22_RESET_PIN * 2));
357     GPIOD->MODER |= (1UL << (CS43L22_RESET_PIN * 2));
358
359     // hold reset low
360     GPIOD->BSRR = (1UL << (CS43L22_RESET_PIN + 16));
361     for (delay = 0; delay < 10000; delay++) {}
362
363     // i2s and i2c init
364     Init_I2S();
365     for (delay = 0; delay < 10000; delay++) {}
366     Init_I2C();
367
368     // release reset
369     GPIOD->BSRR = (1UL << CS43L22_RESET_PIN);
370     for (delay = 0; delay < 10000; delay++) {}
371
372     // cs43l22 configuration
373     I2C_Write_Register(CS43L22_CLK_CTRL, 0x81);
374     I2C_Write_Register(CS43L22_IFACE_CTRL1, 0x04);
375     I2C_Write_Register(CS43L22_IFACE_CTRL2, 0x00);
376
377     // required init sequence (datasheet section 4.11)
378     I2C_Write_Register(0x00, 0x99);
379     I2C_Write_Register(0x47, 0x80);
380     I2C_Write_Register(0x32, 0x80);
381     I2C_Write_Register(0x32, 0x00);
382     I2C_Write_Register(0x00, 0x00);
383     I2C_Write_Register(CS43L22_PWR_CTRL2, 0xAF);
384     I2C_Write_Register(0x08, 0x00);
385     I2C_Write_Register(0x09, 0x00);
386     I2C_Write_Register(0x0A, 0x00);
387     I2C_Write_Register(CS43L22_MISC_CTRL, 0x03);
388     I2C_Write_Register(0x1F, 0x88);
389     I2C_Write_Register(0x06, 0x07);
390     I2C_Write_Register(0x07, 0x07);
391     I2C_Write_Register(0x27, 0x00);
392     I2C_Write_Register(CS43L22_PLAYBACK1, 0xC0);
393     I2C_Write_Register(CS43L22_PLAYBACK2, 0x00);
394     I2C_Write_Register(CS43L22_PCM_VOL_A, 0x00);
395     I2C_Write_Register(CS43L22_PCM_VOL_B, 0x00);
396
397     // power up
398     I2C_Write_Register(CS43L22_PWR_CTRL1, 0x9E);
399
400     for (delay = 0; delay < 10000; delay++) {}
401 }

```

```

402
403 void Audio_Set_Mute(uint8_t mute) {
404     audio_muted = mute;
405 }
406
407 void Audio_Send_Sample(void) {
408     int16_t sample;
409     int16_t amplitude;
410     uint16_t period;
411     int32_t wave_val;
412     uint16_t phase_norm;
413
414     if (audio_muted) { sample = 0; }
415     else {
416         // determine period and amplitude based on tilt
417         period = tilt_period;
418         amplitude = (int16_t)((TONE_AMPLITUDE *
419                               tilt_magnitude) / 100);
420
421         // triangle wave
422         phase_norm = tone_phase % period;
423
424         // calculate wave value
425         if (phase_norm < (period / 2)) {
426             wave_val = (int32_t)amplitude * ((int32_t)phase_norm * 4 -
427                                               (int32_t)period) / (int32_t)period;
428         }
429         else {
430             wave_val = (int32_t)amplitude * ((int32_t)period * 3 -
431                                               (int32_t)phase_norm * 4) / (int32_t)period;
432         }
433
434         // final sample value
435         sample = (int16_t)wave_val;
436     }
437
438     // increment phase
439     tone_phase++;
440     if (tone_phase >= tilt_period) { tone_phase = 0; }
441
442     // send to i2s
443     SPI3->DR = (uint16_t)sample;
444 }
445
446 // =====
447 // spi3 interrupt handler - i2s audio output
448 // =====
449
450 void SPI3_IRQHandler(void) {
451     // check for tx interrupt
452     if (SPI3->SR & SPI_SR_TXE) { Audio_Send_Sample(); }

```

```

453 }
454
455 void Audio_I2C_Write(uint8_t reg, uint8_t value) {
456     // wrapper for i2c write
457     I2C_Write_Register(reg, value);
458 }

```

Thread.h

```

1  // =====
2  // Thread.h - thread declarations and shared data
3  // =====
4
5  #ifndef THREAD_H
6  #define THREAD_H
7
8  #include "cmsis_os.h"
9
10 // =====
11 // system state
12 // =====
13
14 typedef enum { MODE_TILT, MODE_PAUSE_ENTRY, MODE_PAUSE } SystemMode;
15 extern volatile SystemMode current_mode;
16
17 // =====
18 // thread declarations
19 // =====
20
21 extern int Init_Tilt_Thread(void);
22 void Tilt_Thread(void const *argument);
23
24 extern int Init_Button_Thread(void);
25 void Button_Thread(void const *argument);
26
27 extern int Init_LED_Thread(void);
28 void LED_Thread(void const *argument);
29
30 extern int Init_Audio_Thread(void);
31 void Audio_Thread(void const *argument);
32
33 // =====
34 // shared data
35 // =====
36
37 extern volatile int16_t accel_x;
38 extern volatile int16_t accel_y;
39 extern volatile int16_t accel_z;
40
41 #endif

```


Thread.c

```
1 // =====
2 // cmsis-rtos thread definitions
3 // =====
4
5 #include "cmsis_os.h"
6 #include "stm32f4xx.h"
7 #include "my_defines.h"
8 #include "Thread.h"
9
10 // =====
11 // shared variables
12 // =====
13
14 // current system mode
15 volatile SystemMode current_mode = MODE_TILT;
16
17 // accelerometer readings
18 volatile int16_t accel_x = 0;
19 volatile int16_t accel_y = 0;
20 volatile int16_t accel_z = 0;
21
22 // audio control: 1=muted, 0=audible
23 volatile uint8_t audio_muted = 1;
24
25 // tilt-based audio params
26 volatile uint8_t tilt_magnitude = 0;
27 volatile uint16_t tilt_period = 0;
28
29 extern osThreadId tid_Audio_Thread;
30
31 // =====
32 // tilt thread - reads accelerometer
33 // =====
34
35 osThreadId tid_Tilt_Thread;
36 osThreadDef(Tilt_Thread, osPriorityNormal, 1, 0);
37
38 // initialize tilt thread
39 int Init_Tilt_Thread(void) {
40     tid_Tilt_Thread = osThreadCreate(osThread(Tilt_Thread), NULL);
41     if (!tid_Tilt_Thread) return -1;
42     return 0;
43 }
44
45 // tilt thread function
46 void Tilt_Thread(void const *argument) {
47     int16_t x, y, z;
48     int32_t magnitude_sq;
49     uint8_t mag_percent;
50     uint8_t note_index;
```

```

51
52 // a pentatonic scale, 3 octaves from a2 (hz)
53 static const uint16_t scale[] = {
54     110, 131, 147, 165, 196, // a2 c3 d3 e3 g3
55     220, 262, 294, 330, 392, // a3 c4 d4 e4 g4
56     440, 523, 587, 659, 784, // a4 c5 d5 e5 g5
57     880 // a5
58 };
59
60 // blues scale: minor pentatonic + flat5
61 // static const uint16_t scale[] = {
62 //     110, 131, 147, 165, 185, 220, // a2 b2 d3 e3 f#3 a3
63 //     220, 262, 294, 330, 370, 440, // a3 c4 d4 e4 f#4 a4
64 //     440, 523, 587, 659, 740, 880 // a4 c5 d5 e5 f#5 a5
65 // };
66
67 // harmonic minor: a b c d e f g#
68 // static const uint16_t scale[] = {
69 //     110, 123, 147, 165, 196, 220, // a2 b2 d3 e3 g3 a3
70 //     220, 247, 294, 330, 392, 440, // a3 b3 d4 e4 g4 a4
71 //     440, 494, 587, 659, 784, 880 // a4 b4 d5 e5 g5 a5
72 // };
73
74 while (1) {
75     if (current_mode == MODE_TILT) {
76         Read_Accelerometer(&x, &y, &z);
77         accel_x = x; accel_y = y; accel_z = z;
78
79         // tilt magnitude from x and y
80         magnitude_sq = ((int32_t)x * x) + ((int32_t)y * y);
81
82         // scale to 0-100 percent
83         mag_percent = (uint8_t)(magnitude_sq / 2500000L);
84         if (mag_percent > 100) { mag_percent = 100; }
85         if (mag_percent < 20) { mag_percent = 20; }
86         tilt_magnitude = mag_percent;
87
88         // map to scale note (16 notes, even spacing)
89         note_index = ((mag_percent - 20) * 15) / 80;
90         if (note_index > 15) note_index = 15;
91
92         tilt_period = (uint16_t)
93             (BEEP_SAMPLE_RATE / scale[note_index]);
94     }
95     else {
96         // in pause mode, set default audio params
97         tilt_magnitude = 50;
98         tilt_period = BEEP_SAMPLE_RATE / PAUSE_TONE_FREQ_HZ;
99     }
100
101     // sample at defined period

```

```

102         osDelay(ACCEL_SAMPLE_PERIOD_MS);
103         osThreadYield();
104     }
105 }
106
107 // =====
108 // button thread - monitors user button for mode toggle
109 // =====
110
111 osThreadId tid_Button_Thread;
112 osThreadDef(Button_Thread, osPriorityAboveNormal, 1, 0);
113
114 // initialise button thread
115 int Init_Button_Thread(void) {
116     tid_Button_Thread = osThreadCreate(osThread(Button_Thread), NULL);
117     if (!tid_Button_Thread) return -1;
118     return 0;
119 }
120
121 void Button_Thread(void const *argument) {
122     while (1) {
123
124         // switch modes on button click
125         if (Button_Clicked()) {
126             osDelay(200); // debounce
127             if (current_mode == MODE_TILT) {
128                 current_mode = MODE_PAUSE_ENTRY;
129             }
130             else if (current_mode == MODE_PAUSE) {
131                 current_mode = MODE_TILT;
132             }
133         }
134
135         osDelay(50);
136         osThreadYield();
137     }
138 }
139
140 // =====
141 // led thread - controls leds based on mode
142 // =====
143
144 osThreadId tid_LED_Thread;
145 osThreadDef(LED_Thread, osPriorityNormal, 1, 0);
146
147 // initialize led thread
148 int Init_LED_Thread(void) {
149     tid_LED_Thread = osThreadCreate(osThread(LED_Thread), NULL);
150     if (!tid_LED_Thread) return -1;
151     return 0;
152 }

```

```

153
154 void LED_Thread(void const *argument) {
155     int32_t tilt_sq;
156     while (1) {
157
158         // tilt mode: update leds based on tilt
159         if (current_mode == MODE_TILT) {
160             LEDs_All_Off();
161             Update_Tilt_LEDs(accel_x, accel_y);
162
163             // check deadzone for audio
164             tilt_sq = ((int32_t)accel_x * accel_x) +
165                     ((int32_t)accel_y * accel_y);
166
167             // unmute if outside deadzone
168             if (tilt_sq > ((int32_t)TILT_DEADZONE *
169                         TILT_DEADZONE)) {
170                 Audio_Set_Mute(0);
171             }
172             else { Audio_Set_Mute(1); }
173
174             osDelay(50);
175         }
176
177         // pause entry mode: blink all leds once
178         else if (current_mode == MODE_PAUSE_ENTRY) {
179             LEDs_All_On();
180             osDelay(PAUSE_BLINK_ALL_MS);
181             LEDs_All_Off();
182             osDelay(PAUSE_BLINK_ALL_MS);
183
184             current_mode = MODE_PAUSE;
185         }
186
187         // pause mode: blink red led and mute audio
188         else if (current_mode == MODE_PAUSE) {
189             LEDs_All_Off();
190             LED_On(LED_RED);
191             Audio_Set_Mute(0);
192             osDelay(PAUSE_BLINK_RED_MS);
193             LED_Off(LED_RED);
194             Audio_Set_Mute(1);
195             osDelay(PAUSE_BLINK_RED_MS);
196         }
197
198         osThreadYield();
199     }
200 }
201
202 // =====
203 // audio thread - samples via interrupt

```

```

204 // =====
205
206 osThreadId tid_Audio_Thread;
207 osThreadDef(Audio_Thread, osPriorityLow, 1, 0);
208
209 // initialize audio thread
210 int Init_Audio_Thread(void) {
211     tid_Audio_Thread = osThreadCreate(osThread(Audio_Thread), NULL);
212     if (!tid_Audio_Thread) return -1;
213     return 0;
214 }
215
216 void Audio_Thread(void const *argument) {
217     // samples sent by spi3 interrupt at 48khz
218     while (1) { osDelay(1000); }
219 }

```